

PRD: Admins Can Add Situational Tests to Courses

1. Overview

This ticket introduces **Situational Tests** as a new learning object type in the course builder. A situational test is a scenario-based assessment: a written brief describing a realistic situation, followed by one or more multiple-choice questions that test the learner's judgement within that scenario. Admins will create and manage these from the existing "Add Content" side drawer in the course builder, following the same interaction patterns already established for assessments.

The work is scoped to **Phase 1: Admin create/edit UI only**. Phase 2 (AI-assisted generation of situational tests from lesson content) is a future iteration and is explicitly out of scope. A learner-facing rendering experience already exists, so no learner UI work is required.

The decision to model situational tests as a distinct learning object type — rather than a new assessment sub-type — avoids backend complexity while leveraging existing infrastructure. The backend already supports this object type; the gap is purely in admin-facing authoring UI.

2. Jobs To Be Done

Main Job

When I am building a course, I want to add scenario-based assessments that test learners against realistic situations, so I can evaluate their applied judgement — not just factual recall.

Job Map

Stage	What the Admin Does	Current Pain / Workaround
Define	Decides a realistic scenario is needed to test applied knowledge	No dedicated format exists; admins either skip scenario-based testing or hack it by writing long preambles inside individual assessment question descriptions
Locate	Finds the right place in the course outline to insert the test	The "Add Content" sidebar exists but has no situational test option; admins must improvise with regular assessments
Prepare	Writes the scenario brief that sets the context	Currently must copy-paste the same scenario text into every individual question's description — duplication and drift risk
Confirm	Reviews that the brief and all questions make sense as a coherent bundle	No way to view a scenario + its questions as a unit; each question is a standalone object

Execute	Adds multiple-choice questions tied to the scenario	Must create separate assessment items one by one, with no grouping mechanism
Monitor	Checks that all questions are complete and the test is ready	No unified view of a situational test; admin must mentally track which assessments belong to which scenario
Resolve	Edits questions or the brief after initial creation	Editing the scenario means finding and updating every individual question that carried the duplicated brief text

Related Jobs

- **Curate course structure:** Admins arrange learning objects in a meaningful pedagogical sequence. Situational tests must fit naturally into this flow.
- **Ensure assessment quality:** Admins need confidence that scenario-based questions are well-formed before learners encounter them.
- **Prepare for AI-assisted authoring (future):** Phase 2 will let AI generate situational tests from lesson content. Phase 1 UI choices should not block this.

Emotional & Social Dimensions

- **Confidence:** Admins want to feel that the tool helps them create rigorous, professional assessments — not just bare-bones quizzes.
- **Efficiency:** They want to move quickly through course building without fighting the tool.
- **Credibility:** Courses with scenario-based assessments signal pedagogical sophistication to stakeholders and learners.
- **Control:** Admins want to be the author, not fight a complex system. A simple, predictable flow respects their expertise.

3. Goals

1. **Parity with existing patterns:** Adding a situational test should feel as familiar as adding an assessment — same drawer, same interaction conventions, minimal learning curve.
2. **Scenario-as-container:** The scenario brief is the organising unit. All questions are visually and structurally grouped under it, eliminating the duplication workaround.
3. **Speed to value:** An admin can create a complete situational test (brief + 2–3 questions) in under 3 minutes.
4. **Phase 2 readiness:** The data model and UI surface should accommodate future AI-generated content without requiring structural rework.
5. **Simplicity:** No draft/published states, no duplicate/archive features, no scoring configuration beyond what already exists at the course level.

4. Job Stories

Course Authoring

- **When** I am building a course and reach a module where learners need to apply knowledge to a realistic scenario, **I want to** add a situational test directly from the content sidebar, **so I can** keep my authoring flow uninterrupted.
- **When** I open the "Add Content" sidebar, **I want to** see "Situational Test" as a clearly labelled option positioned before "Assessments," **so I can** quickly find and select it.

Scenario Brief Authoring

- **When** I start creating a situational test, **I want to** write the scenario brief first as a dedicated step, **so I can** focus on crafting a clear, realistic situation before worrying about questions.
- **When** I'm writing a scenario brief and feel uncertain about what to write, **I want to** see placeholder or example text, **so I can** overcome the blank-page problem and understand the expected format.

Question Authoring

- **When** I have saved my scenario brief, **I want to** add multiple-choice questions one by one within the same drawer, **so I can** build the test incrementally without losing sight of my scenario.
- **When** I am adding answer options to a question, **I want to** clearly mark which option is correct, **so I can** ensure accurate scoring without ambiguity.
- **When** I have added one question, **I want to** add another question to the same scenario, **so I can** create a comprehensive situational test with multiple angles on the same situation.

Editing

- **When** I return to a previously created situational test, **I want to** edit the brief or any question, **so I can** refine content based on feedback or new information.
- **When** I need to remove a question that no longer fits the scenario, **I want to** delete it individually without affecting the rest of the test, **so I can** maintain a clean, relevant assessment.

Course-Level Context

- **When** I look at the course outline, **I want to** see situational tests displayed as distinct learning objects, **so I can** understand the course structure at a glance and know where scenario-based assessments appear.

5. Requirements

Functional Requirements

FR-1: Content Sidebar Entry Point

- A new "Situational Test" option must appear in the "Add Content" sidebar within the course builder.
- It must be positioned immediately before the existing "Assessments" option.
- It must use a distinct icon and label to differentiate it from standard assessments.

FR-2: Two-Step Side Drawer Creation Flow

- Selecting "Situational Test" opens a side drawer following the existing "Add assessment" drawer pattern.
- **Step 1 — Scenario Brief:**
 - A text field (rich text or plain text, matching existing assessment description fields) for the scenario brief.
 - The brief is mandatory; the admin cannot proceed to Step 2 without entering content.
 - Placeholder/example text should guide the admin (e.g., *"Describe a realistic situation the learner might face..."*).
 - A descriptive action button (e.g., "Save Brief & Add Questions") advances to Step 2.
 - **Authoring scaffold (see FR-11).** The brief field must prompt the four-part structure rather than presenting a blank box.
 - **Length guidance.** A live word count with a soft target of **80 words or fewer**. Over-length is a warning, not a block — the admin can still save.
- **Step 2 — Questions:**
 - The scenario brief is visible (collapsed or summary) at the top for reference.
 - An "Add Question" action lets the admin add multiple-choice questions one at a time.
 - Each question consists of: question text, 2 or more answer options, and a correct-answer indicator (radio toggle or equivalent).
 - Admin can add multiple questions within the same step.
 - A "Save" / "Done" action closes the drawer and adds the situational test to the course outline.

FR-3: Multiple-Choice Question Format

- Only multiple-choice (single correct answer, radio-select) is supported.
- Each question must have a minimum of 2 answer options.
- No maximum constraint on answer options or questions per test is enforced by the UI (follow backend constraints if any).

- The correct answer must be explicitly marked by the admin before saving.
- **Length guidance.** Soft target of **25 words or fewer per response option**, surfaced the same way as the brief's word count — a warning, never a block.
- **One skill per question.** Each question should assess a single skill tied to the scenario. Surfaced as authoring guidance, not validated by the system.

FR-4: Editing

- Admins can reopen an existing situational test from the course outline.
- The drawer reopens in an editable state showing the brief and all questions.
- Admins can: edit the scenario brief, edit any question's text or answer options, change the correct answer, delete individual questions, and add new questions.
- No reordering of questions is required for Phase 1 (nice-to-have, not mandatory).

FR-5: Deletion

- Admins can delete an entire situational test from the course outline, following the same delete pattern as other learning objects.
- Confirmation prompt required before deletion.

FR-6: Course-Level Scoping

- A situational test belongs to the course it was created in. It cannot be reused or shared across courses.

FR-7: Scoring

- Each question has exactly one correct option; all others score zero. There is no partial credit.
- No per-test scoring configuration in this drawer. The pass threshold is a percentage the admin sets in the course's existing pass-score settings, and applies to the situational test the same way it applies to other assessments.
- Out of scope for this ticket: any change to how the pass score is configured or calculated.

FR-8: Validation & Error Handling

- Inline validation on field blur (not on submit) for required fields.
- Clear error messages: e.g., "Please enter a scenario brief," "Each question needs at least 2 answer options," "Please mark a correct answer."
- The admin must not be able to save a question without a correct answer marked.

FR-9: Course Outline Representation

- Situational tests appear in the course outline as distinct learning objects with a recognisable icon and the scenario brief title (or first N characters) as the label.
- They are positioned in the outline wherever the admin placed them.

FR-10: Phase 2 Affordance (Stub)

- No AI-generation UI is built in Phase 1.
- However, the data model and drawer layout should not preclude adding an "AI Generate" action in Step 2 in a future iteration. This is an architectural consideration, not a UI requirement.

FR-11: Authoring Guidance — Writing a Good Situational Test

The quality of a situational test is set almost entirely at authoring time, and the failure mode is well documented: scenarios that carry extraneous detail don't just slow the learner down, they introduce doubt about which information is even relevant to the decision. The product should make the good version the easy version.

Scenario brief — the four-part scaffold. The brief field guides the admin through four parts, in order. This is presented as inline prompts or a structured placeholder, *not* four separate inputs — the saved brief is a single flowing paragraph:

Part	What it establishes	Example
1. Position of the subject	Who the learner is in this scenario, so they read the rest correctly	<i>"You work for a company that makes plastic bottle tops. It's your job to secure raw material for production."</i>
2. The situation	The context, directly tied to the skill being tested	<i>"You are negotiating with a supplier to sell you dye so you can colour your plastics."</i>
3. The complication	The crux — the specific thing that must be responded to	<i>"You know this supplier has the best dyes, but another company sells acceptable lower-quality dyes as a Plan B."</i>
4. The question and goal	What to decide, and the outcome a correct answer should achieve	<i>"What negotiation tactic will get you the best price?"</i>

Naming the goal (not just "What should you do?") is what makes the response options definable — without it, several options look equally defensible.

Response options — the rules. Options must be *actions the learner could take*, never the *outcomes* of actions. Beyond that they should:

- share the same grammatical structure;
- be related in content — different, but different in the same way;
- follow logically from the scenario and the question asked;
- introduce no context absent from the brief;
- remain plausible even when wrong. A visibly absurd option tests nothing.

Where this lives in the UI. Per the Built for Mars finding on contextual education (§6), this guidance is bound to the moment of need — prompts and examples inside the brief and option fields — rather than front-loaded as a wall of instructions the admin skips.

6. Research & Best Practices

Synthesised Findings

The two-step drawer approach (brief → questions) is well-supported by established UX patterns. **NN/g's wizard guidelines** recommend that each step be self-sufficient — admins should not need to leave the drawer to retrieve information — and that button labels be descriptive (e.g., "Save Brief & Add Questions" rather than a generic "Next"). They also caution against modal windows that block context the user might need, which favours an inline or non-blocking drawer variant ([NN/g — Wizards](#)).

PatternFly's drawer design guidelines distinguish between overlay drawers (blocking) and inline drawers (content-pushing). Since admins may want to reference the course outline while adding a situational test, an inline drawer — or at minimum an overlay that doesn't fully obstruct the outline — is preferable ([PatternFly — Drawer Guidelines](#)).

UX Planet's side panel guidance reinforces the drawer anatomy: a clear header with title and close button, a scrollable body, and a sticky footer for primary actions. This directly maps to the planned drawer layout ([UX Planet — Side Panels](#)).

Growform's multi-step form data shows that splitting forms with 7+ fields into multiple steps increases completion by up to 86%. A situational test with brief + multiple questions easily exceeds 7 fields, validating the two-step approach. They emphasise progress indicators, inline validation on blur, and top-aligned labels — all applicable here ([Growform — Multi-Step Form Best Practices](#)).

The domain model is validated by **TestGorilla's SJT format** — one scenario brief followed by multiple pre-scored multiple-choice questions — which maps directly to the DES-276 data model ([TestGorilla — Writing SJT Questions](#)). The **U.S. OPM's assessment methodology** further confirms: SJTs present one scenario with multiple questions about that scenario, always multiple-choice, no free-text ([U.S. OPM — Situational Judgment Tests](#)).

Sana Learn has no equivalent, and how it falls short is instructive. Sana splits the two halves of a situational test across two features that never meet. Its **Assessment Card** groups several questions into one container with shared settings — time limit, retries, a single "Correct my answers" submit — but carries no scenario brief across them. Its **Scenario Card** is named for scenarios but is a different product entirely: an AI counterpart the learner role-plays against by chat or voice, with generated feedback rather than scored options. Its question types are *Pick the best option*, *Select all that apply*, *True or false* and *Match the pairs* — none of which bundle under a shared situation ([Sana — Questions and Assessments](#); [Sana — Interactive Cards](#)). The combination DES-276 describes — one written brief, several multiple-choice questions scored against it — is therefore not a copy of an existing competitor feature.

Among competitor platforms, **Kajabi's model** of treating quizzes as separate lesson objects in a sidebar is the closest analog to the planned approach. Kajabi's admin workflow — create quiz shell → add questions one by one → set grading — mirrors the planned two-step drawer ([Zapier — Kajabi vs Teachable](#); [Course Platform Comparison](#)). Thinkific's richer assessment suite (quiz, exam, survey, assignment) shows the direction the platform could evolve, but its complexity is beyond Phase 1 scope.

UX References

App	Flow / Screen	URL	Pattern Description	Relevance
Kajabi	Course builder — add content side panel	View	Side drawer with content type selection in course builder	Direct competitor pattern — shows how a sidebar lists learning object types, exactly where "Situational Test" will be added
Teachable	Course builder — quiz/assessment editing	View	Inline assessment editing within course curriculum	Shows how assessment editing can live within the course builder context without full-page navigation
Kajabi	Multiple choice question builder form	View	Question authoring form with answer options	Reference for MCQ creation UI: question text field + answer option list with correct answer toggle
Teachable	Question builder with scenario/description	View	Assessment question editing with description area	Shows how a question + description text area can be combined — relevant to the scenario brief + question pattern
Podia	Multiple choice question form with options	View	MCQ builder with radio-button answer selection	Clean, minimal question builder pattern with inline answer options and correct-answer marking
Typeform	Question builder with answer choices	View	Form builder with multiple choice options	Typeform's approach to inline editing of questions and answer choices — polished interaction model
Whop	Course content sidebar with object types	View	Content type picker sidebar in course builder	Shows sidebar listing of available learning object types — comparable to the "Add Content" panel
Teachable	Course outline with content type sidebar	View	Course structure view with add-content sidebar	Shows spatial relationship between course outline and content-adding panel
Coursera	Course content sidebar with learning objects	View	Course builder with content type selection	Enterprise LMS example of structured content type sidebar — validates the pattern at scale

Deel	Admin form in side drawer panel	View	Side drawer with multi-field form	SaaS pattern for complex form entry in a drawer overlaying main content — applicable to the brief + questions form
Workable	Multi-step form in side drawer	View	Side drawer with structured form steps	HR SaaS showing a multi-step creation flow inside a side panel — closest structural analog to the two-step drawer
Asana	Task creation side panel	View	Side panel for creating/editing structured content	Canonical SaaS pattern for creating items in a side panel without leaving the main view

Built for Mars Lessons

Title	Type	URL	Lesson	Relevance
Progressive Disclosure	Glossary	View	The <i>information horizon</i> — what a user can see at any moment — should be carefully controlled. If too broad, users feel overwhelmed. Each step should reveal only the fields relevant to that phase.	Directly validates the two-step drawer: Step 1 shows only the scenario brief; Step 2 reveals question authoring. Prevents cognitive overload.
Revolut's Progressive Onboarding	UX Bite	View	Education should be bound to the moment it's relevant, not front-loaded. Contextual hints at the point of need outperform upfront instruction.	Supports providing contextual guidance (e.g., "Add at least 2 answer options") in Step 2 rather than explaining all rules upfront in Step 1.
Why Adding Friction Improved BFM's Sign-Up	Case Study	View	Strategic friction improves outcomes — adding steps that set expectations increased conversions 25%. Not all friction is bad; friction that ensures quality is valuable.	Validates requiring the scenario brief as a mandatory prerequisite before question authoring. This is intentional friction that ensures admins establish context first, improving test quality.
How to Stop People from Skipping Your Onboarding (YNAB)	Case Study	View	Visible progress, early value, and sensible defaults prevent abandonment in multi-step flows.	Applies to preventing admin abandonment: show a step indicator (Step 1 of 2 / Step 2 of 2), provide placeholder text to reduce blank-page anxiety,

				and allow saving a partially complete test.
--	--	--	--	---

7. Plan of Action

Phase 1A: Design & Specification

- ☐ Audit the existing "Add Assessment" drawer interaction (open/close behaviour, field types, button placement, validation patterns) to establish the baseline pattern to replicate.
- ☐ Create wireframes for the "Add Content" sidebar showing "Situational Test" in its new position (immediately before "Assessments").
- ☐ Design the Step 1 drawer (Scenario Brief): text field with placeholder text, mandatory validation, "Save Brief & Add Questions" button, step indicator (1/2).
- ☐ Design the Step 2 drawer (Questions): collapsed/summary brief at top, "Add Question" action, MCQ authoring fields (question text, answer options, correct-answer toggle), "Add Another Question" action, "Save" / "Done" footer button, step indicator (2/2).
- ☐ Design the edit state: drawer reopens showing brief + questions in editable form, individual question deletion with confirmation.
- ☐ Design the course outline representation: icon, label (truncated brief), and positioning behaviour.
- ☐ Design validation and error states: inline validation on blur, error messages for empty brief, insufficient answer options, missing correct answer.
- ☐ Conduct internal design review with the product and engineering team.

Phase 1B: Frontend Implementation

- ☐ Add "Situational Test" entry to the "Add Content" sidebar component with icon and label.
- ☐ Implement Step 1 drawer: scenario brief text input with validation, step indicator, and "Save Brief & Add Questions" action.
- ☐ Implement Step 2 drawer: brief summary display, question authoring form (question text + answer options + correct-answer toggle), "Add Question" / "Add Another Question" actions, and "Save" / "Done" action.
- ☐ Implement edit flow: open existing situational test in drawer with pre-populated data, support editing brief, editing/deleting/adding questions.
- ☐ Implement delete flow: delete entire situational test from course outline with confirmation dialog.
- ☐ Implement course outline rendering: situational test as a distinct learning object with icon and truncated brief as label.
- ☐ Implement inline validation and error messaging per FR-8.

Phase 1C: Backend Integration

- ☐ Confirm existing API contract supports CRUD operations for situational test learning objects (create, read, update, delete).
- ☐ Integrate frontend with backend endpoints for creating a situational test (brief + questions payload).
- ☐ Integrate frontend with backend endpoints for updating (editing brief, adding/editing/deleting questions).
- ☐ Integrate frontend with backend endpoint for deleting a situational test.
- ☐ Validate that course-level scoring correctly incorporates situational test questions.

Phase 1D: QA & Polish

- ☐ Test the full create flow: add situational test from sidebar → write brief → add questions → save → verify in course outline.
- ☐ Test the edit flow: reopen → edit brief → edit/add/delete questions → save.
- ☐ Test the delete flow: delete situational test → confirm → verify removal from outline.
- ☐ Test validation edge cases: empty brief, question with < 2 options, question with no correct answer marked, very long text inputs.
- ☐ Test that existing assessment and learning object flows are unaffected (regression).
- ☐ Verify that the learner-facing view correctly renders admin-created situational tests.
- ☐ Cross-browser and responsive testing for the drawer component.

Future: Phase 2 — AI Generation (Out of Scope)

- ☐ Add "AI Generate" action in Step 2 of the drawer that auto-generates questions from lesson content, course name, and scenario brief.
- ☐ Design review and approval flow for AI-generated questions before they are added to the test.

8. Risks & Mitigations

Risk	Impact	Mitigation
Existing "Add Assessment" drawer pattern has undocumented complexity — the new drawer may diverge from the established pattern, creating inconsistency	Medium — confuses admins who expect uniform behaviour	Audit the existing drawer thoroughly in Phase 1A; use the same underlying component/template where possible

Backend API contract assumptions are wrong — the ticket states "backend already supports it" but the exact contract is unconfirmed	High — frontend work may need rework if API shape differs from assumptions	Confirm API contract with backend team before starting Phase 1B; create an API integration spike task early
No question reordering may frustrate admins — admins who create many questions per scenario may want to reorder them	Low — Phase 1 constraint is acceptable for initial release	Document as a known limitation; plan to add drag-to-reorder in a fast-follow if user feedback warrants it
Unbounded questions/options could create performance or UX issues — no caps on questions per test or options per question	Medium — extremely long tests could degrade drawer performance or create confusing learner experiences	Monitor usage; if needed, add soft warnings (e.g., "You have 20+ questions — consider splitting into multiple tests") in a future iteration
Phase 2 AI generation may require data model changes — if the current model doesn't accommodate AI metadata (source lesson, confidence scores), Phase 2 could be disruptive	Medium — could require migrations or refactoring	Ensure the data model includes extensible metadata fields (e.g., a JSON blob or tags field) even if unused in Phase 1
Learner-facing view compatibility — the existing learner view may not handle edge cases from the new admin authoring flow (e.g., single-question tests, very long briefs)	Medium — broken learner experience undermines the feature	Include learner-view rendering verification in Phase 1D QA; coordinate with the team that owns the learner view
Admin abandonment during two-step flow — admins may start a test but not finish it, leaving orphaned partial records	Low — depends on save behaviour	Implement auto-save or a clear "discard" action; ensure the backend handles partial records gracefully (either save as incomplete or prevent orphaned data)